

Интеграција модула, уговори и подаци

EquipmentReservation као извршива основа за интеграцију модула и наставак развоја.

Заједнички извршиви пример

EquipmentReservation се надограђује кроз интеграцију, AI радни ток, MCP и guardrails.

Студент прати како се исти систем развија кроз интеграцију, AI workflow, MCP и guardrails.

НАПОМЕНА

EquipmentReservation.sln је главна улазна тачка наставног примера.

Покретање и провера наставног примера

Прва провера обухвата restore, build и test команде над solution-ом.



```
1 cd examples/ers-ai-workflow
2 dotnet restore EquipmentReservation.sln
3 dotnet build EquipmentReservation.sln -c Release --no-restore
4 dotnet test EquipmentReservation.sln -c Release --no-build
```

НАПОМЕНЕ

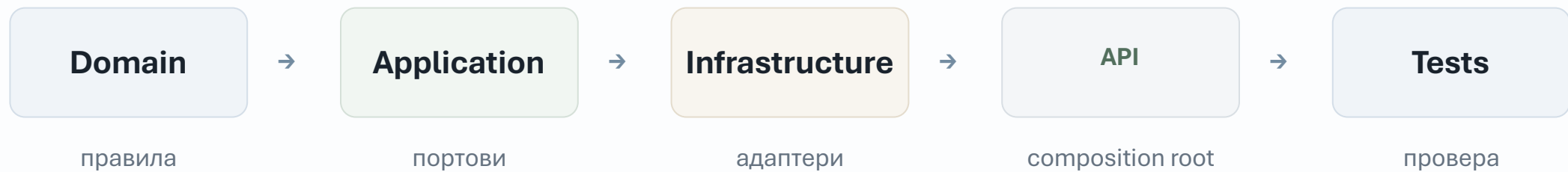
Радимо над целим solution-ом.

Команде су део доказа, не декоративан додатак.

Пад build-а или теста мора се решити пре даље интеграције.

Структура solution-а и смер зависности

MCP и Guardrails припадају истом solution-у, без увођења зависности у пословно језгро.



НАПОМЕНА

Смер зависности има предност над физичким распоредом директоријума.

Одговорности пројеката у solution-у

Сваки пројекат има јасно дефинисану одговорност.

Пројекат	Одговорност
Domain	ентитети и пословна правила; без пројектних зависности
Application	use-case-ови и портови према другим модулима
Infrastructure	имплементације портова
Api	composition root и HTTP граница
Mcp	контролисано излагање контекста AI клијенту
Guardrails	детерминистичке политике за ризичне позиве
Tests	NUnit провере домена, use-case-a и guardrail-a

Пословно правило у доменском сервису

`InventoryItem` представља стање и нема зависности од HTTP-а, базе, MCP-а или AI алата.

`InventoryReservationService` примењује правило: количина мора бити позитивна и није дозвољено резервисати више јединица од расположиве количине.

НАПОМЕНА

Исто правило користе API, тестови и будући адаптери без дуплирања.

Валидација захтева

Валидатор проверава облик и опсег улаза пре покретања пословног тока.

```
1 public static void ValidateAndThrow(  
2     CreateReservationCommand command)  
3 {  
4     ArgumentNullException.ThrowIfNull(command);  
5     if (command.RequestId == Guid.Empty)  
6         throw new ArgumentException(  
7             "Request id is required.");  
8     if (command.Quantity <= 0)  
9         throw new ArgumentOutOfRangeException(  
10             nameof(command));  
11 }
```

НАПОМЕНЕ

Модел само носи стање.

Validator проверава обавезна поља и опсег.

Handler покреће проверу пре позива портова.

Уговор између модула Reservations и Inventory

Апликациони слој описује минималне операције потребне случају употребе.

Не прослеђује ORM ентитет, не отвара интерно складиште и не крши границу другог модула.

НАПОМЕНА

Уговор између модула примењује ISP и DIP.

Порт апликационог слоја

Случај употребе зависи од минималног уговора који му је потребан.

```
1 public sealed record ReserveInventoryRequest(  
2     Guid EquipmentId, int Quantity, Guid ReservationId);  
3  
4 public sealed record ReserveInventoryResult(  
5     bool Success, string? ErrorCode);  
6  
7 public interface IInventoryModule  
8 {  
9     Task<ReserveInventoryResult> ReserveAsync(  
10         ReserveInventoryRequest request,  
11         CancellationToken cancellationToken);  
12 }
```

НАПОМЕНЕ

Порт је у Application слоју.

Infrastructure реализује уговор.

Use-case не зна интерне детаље Inventory модула.

Handler случаја употребе

Handler валидира улаз, проверава поновљени захтев и позива Inventory преко порта.

```
1 CreateReservationCommandValidator
2   .ValidateAndThrow(command);
3   await using var lease = await _requestLock
4   .AcquireAsync(command.RequestId, ct);
5   var existing = await _reservations
6   .FindByRequestIdAsync(command.RequestId, ct);
7   if (existing is not null)
8     return Map(existing, replayed: true);
9   var inventoryResult = await _inventory.ReserveAsync(
10    new(command.EquipmentId, command.Quantity,
11      reservationId), ct);
12   var status = inventoryResult.Success
13     ? ReservationStatus.Confirmed
14     : ReservationStatus.Rejected;
15   var reservation = new Reservation(
16     reservationId, command.RequestId,
17     command.EquipmentId, command.StudentId,
18     command.Quantity, status,
19     inventoryResult.ErrorCode);
20   await _reservations.AddAsync(reservation, ct);
21   return Map(reservation, replayed: false);
```

НАПОМЕНЕ

Модел нема пословне методе.

Inventory правило остаје иза порта.

Application враћа стабилан исход API слоју.

Idempotency key и поновљени захтев

Поновљени захтев са истим идентификатором не производи додатни ефекат.

RequestId

један захтев, један ефекат

Ако исти захтев стигне поново, постојећа резервација се враћа без другог позива на промену стања.

Тест идемпотентности

Тест потврђује да поновљени позив не мења стање други пут.

```
1 var first = await handler.HandleAsync(command, ct);
2 var replay = await handler.HandleAsync(command, ct);
3 var available = await inventory.GetAvailableAsync(
4     command.EquipmentId, ct);
5
6 Assert.That(first.Outcome,
7     Is.EqualTo(CreateReservationOutcome.Confirmed));
8 Assert.That(replay.ReservationId,
9     Is.EqualTo(first.ReservationId));
10 Assert.That(replay.Replayed, Is.True);
11 Assert.That(available, Is.EqualTo(3));
```

НАПОМЕНЕ

Тест проверава уговор између модула.

Поновљени позив не сме да направи нови side effect.

Проверавају се и исход и преостала количина.

Докази интеграције

Build, тестови, diff и уговори потврђују исправност интеграције.

01

Build

целокупан solution се компајлира

02

Tests

пословни и интеграциони сценарији пролазе

03

Diff

промена је ограничена на очекиване слојеве

04

Contract

уговор између модула је стабилан

API као composition root

HTTP граница саставља зависности и мапира резултат без пословних одлука.

API РАДИ

прима HTTP захтев

мапира DTO у команду

позива handler

мапира Result у одговор

API НЕ РАДИ

не проверава залиху директно

не мења Domain стање мимо use-case-a

не чита интерну базу другог модула

не садржи пословну одлуку

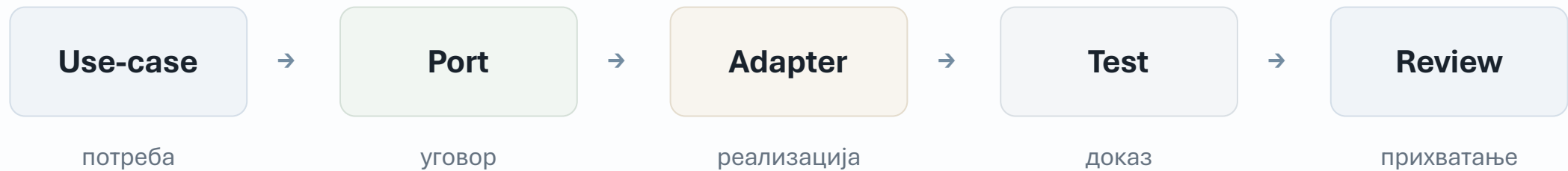
Ризици интеграције

Сваки ризик повезује се са одговарајућом провером.

Ризик	Провера
Погрешан смер зависности	project references и build
Дуплирана правила у API-ју	code review и diff
Поновљен захтев прави нови ефекат	idempotency тест
Порт пропушта интерне детаље	преглед Application уговора
MCP/Guardrails улазе у Domain	dependency rule провера

Интеграција модула у заједнички производ

Модули сарађују преко уговора, адаптера, тестова и прегледа кода.



НАПОМЕНА

Модули се повезују преко стабилних уговора са јасним пословним значењем.

Границе модула и AI радни ток

Стабилне границе ограничавају контекст, обим измене и поступак провере.

01

Контекст

погођени пројекти су
познати

02

Ограничења

зна се шта не сме да се
мења

03

Провера

build, test и diff су довољни
сигнали

Задатак на вежби

EquipmentReservation се користи као заједнички извршиви пример.

- 01 Отворити EquipmentReservation.sln.
- 02 Проверити build и test команде.
- 03 Пронаћи Domain правило и његов Application порт.
- 04 Објаснити како CreateReservationHandler користи Inventory.
- 05 Показати тест који штити идемпотењтност или уговор између модула.

Резиме

Интеграција повезује модуле преко стабилних уговора уз очуване архитектонске границе.

Добра интеграција значи да модули сарађују преко јасних уговора, док пословно језгро остаје заштићено од детаља транспорта, складишта и развојних алата.

НАПОМЕНА

Наредна вежба уводи контролисан AI радни ток.

Припрема за наредну вежбу

Припремити solution, тестове и границе модула за контролисан AI радни ток.

- 01 Забележити како се solution покреће и тестира.
- 02 Описати границе Domain, Application, Infrastructure и API пројеката.
- 03 Изабрати један мали захтев за промену правила.
- 04 Навести фајлове које промена сме да погађа.
- 05 Припремити проверу build/test/diff за следећу вежбу.